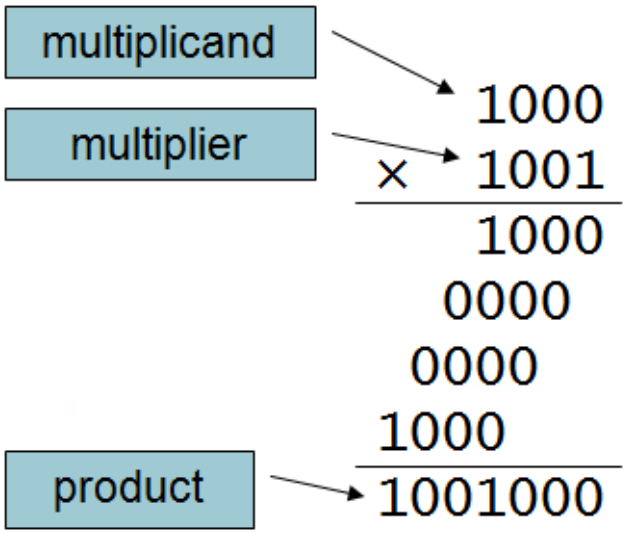
Computer Architecture
Arithmetic for Computers

Tatpong Katanyukul

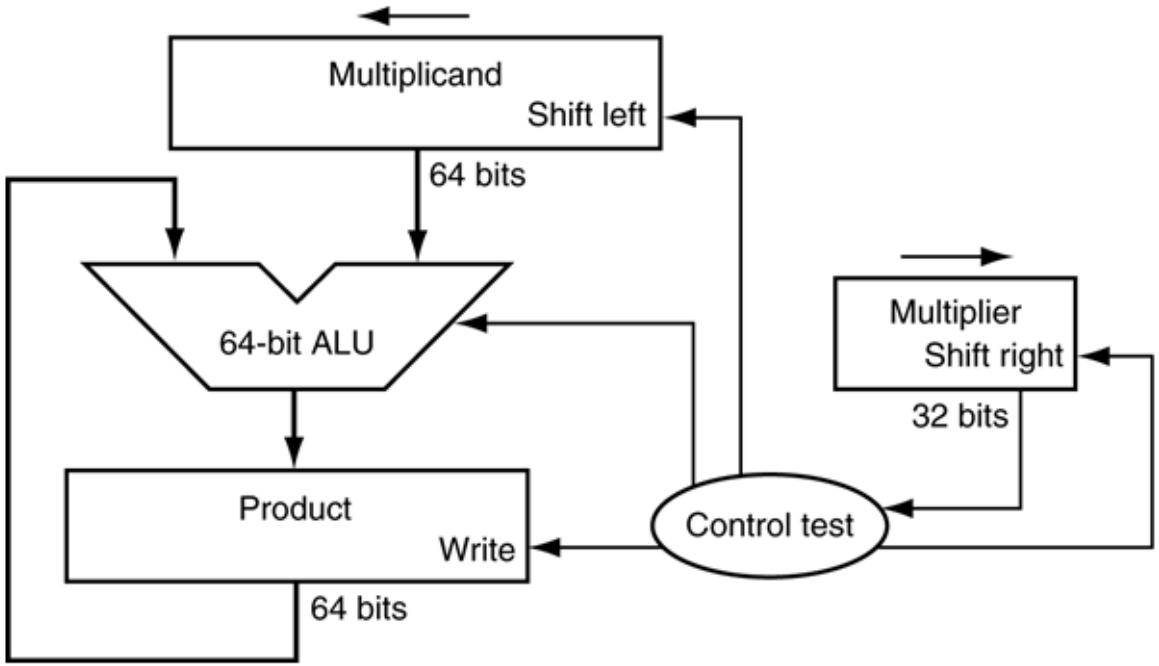
CoE, KKU

20 มีนาคม พ.ศ. 2560

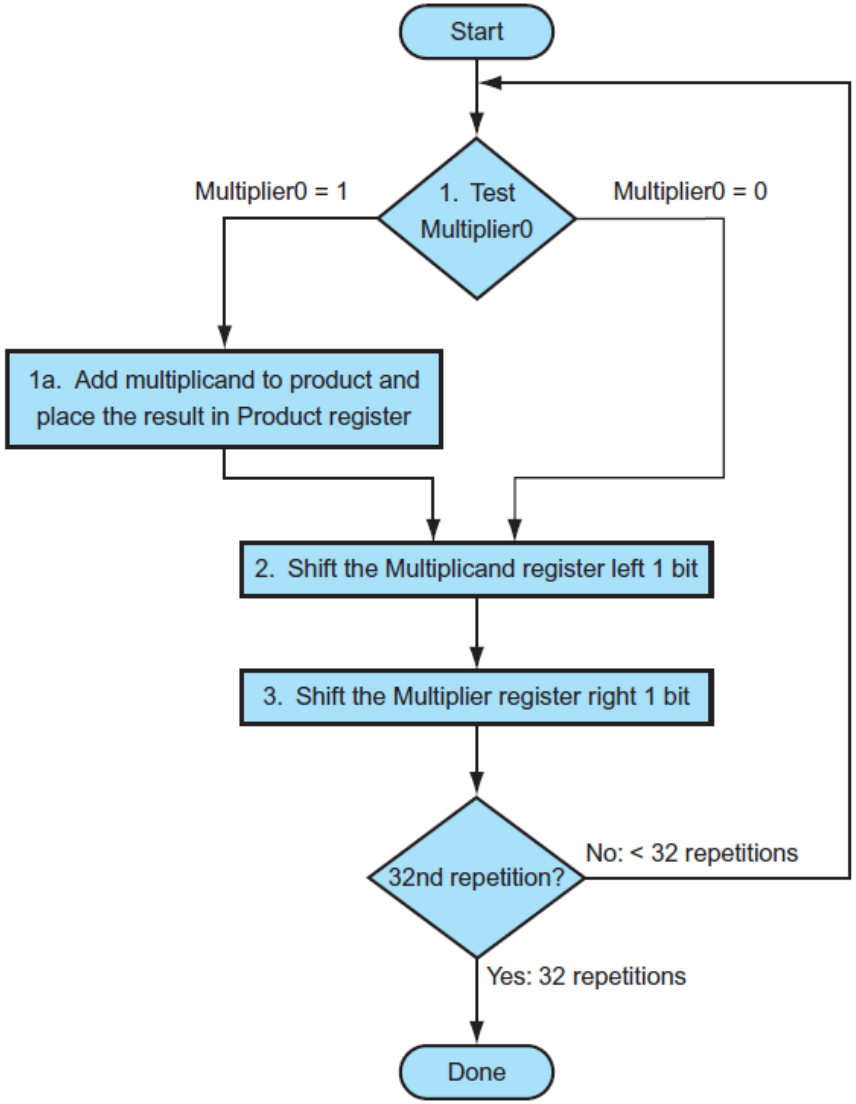
Integer Multiplication: Approach 1

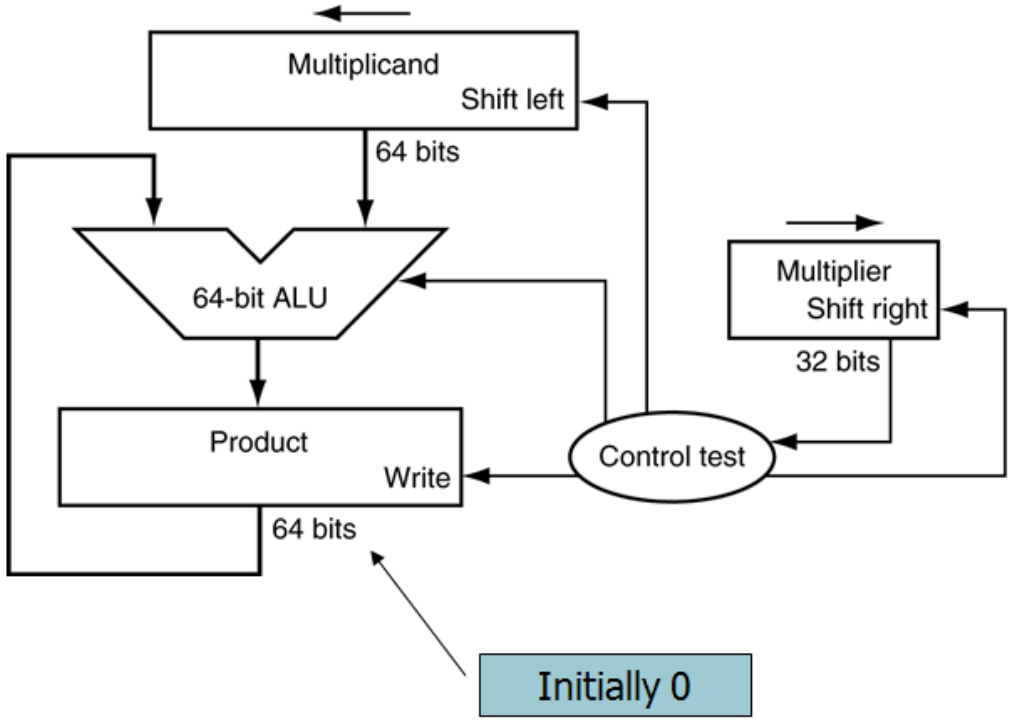


Length of product is the sum of operand lengths

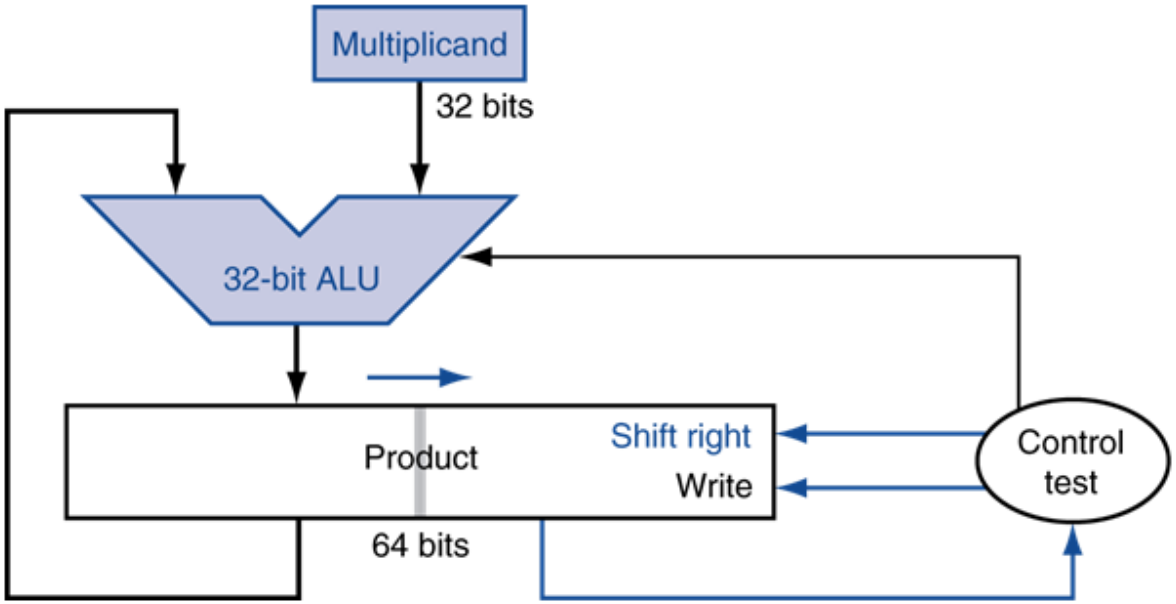


Integer Multiplication: Approach 1



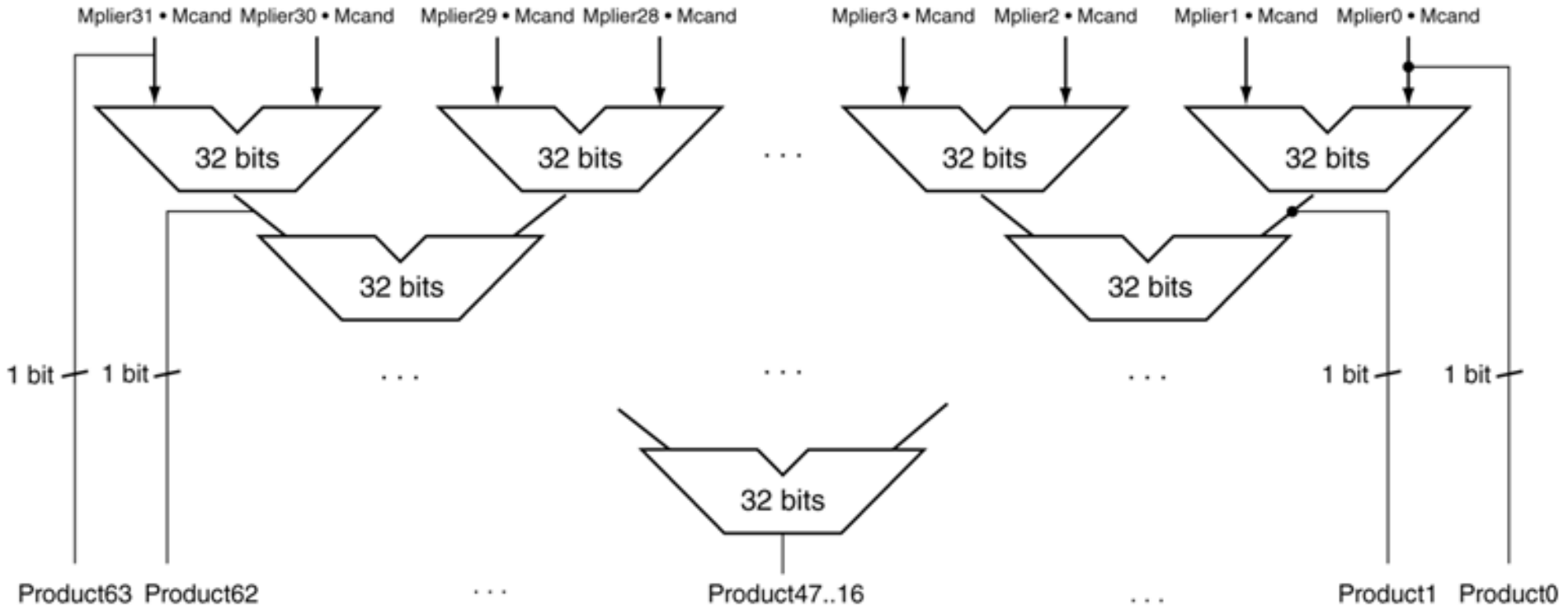


Integer Multiplication: Approach 2



- Load multiplier on the right half of Product
- If Muliplier0 = 1, do add and shift; otherwise just do shift.
- One cycle per partial-product addition

Multiplication: Fast Multiplication



- Cost/performance tradeoff

MIPS Multiplication

- Two 32-bit registers for product
  - HI: most-significant 32 bits
  - LO: least-significant 32-bits
- Instructions
  - mult rs, rt (signed version) or multu rs, rt (unsigned version)

    64-bit product in HI/LO
  - mfhi rd / mflo rd

    Move from HI/LO to rd

    Can test HI value to see if product overflows 32 bits
  - mul rd, rs, rt

    Least-significant 32 bits of product $\rightarrow$ rd

Hand-On: Signed v.s. Unsigned Multiplication

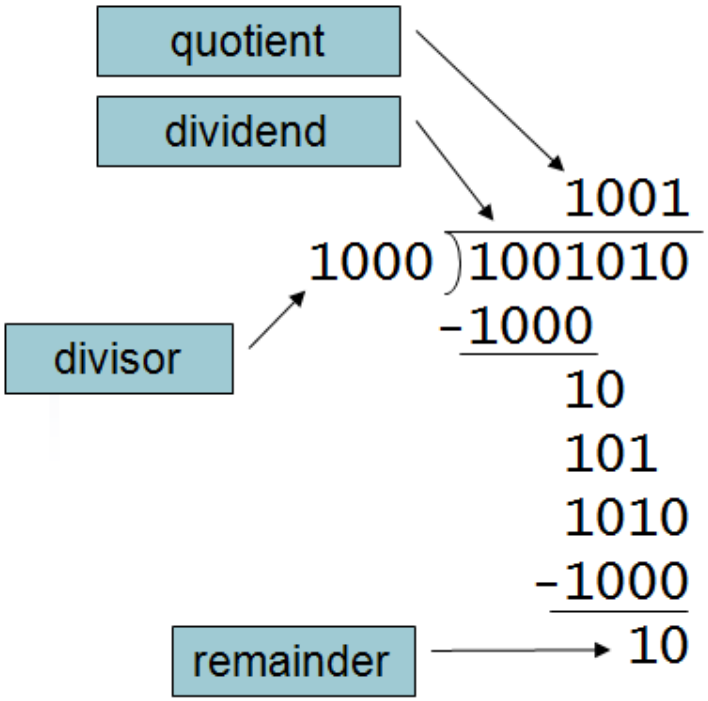
0xfffffffb = -5 (signed) / 4,294,967,291 (unsigned)

```
.text
.globl main
main:
    li   $s1, 0xfffffffb
    li   $s2, 0x00000002
    mult $s1, $s2
    mfhi $s3
    mflo $s4
    nop
```

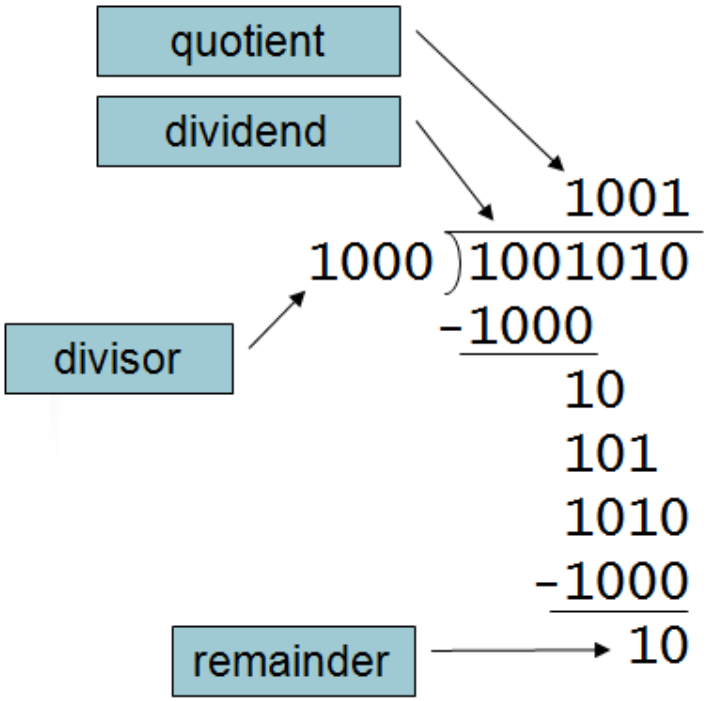
```
.text
.globl main
main:
    li   $s1, 0xfffffffb
    li   $s2, 0x00000002
    multu $s1, $s2
    mfhi $s3
    mflo $s4
    nop
```

What do you get?

Integer Division: Approach 1

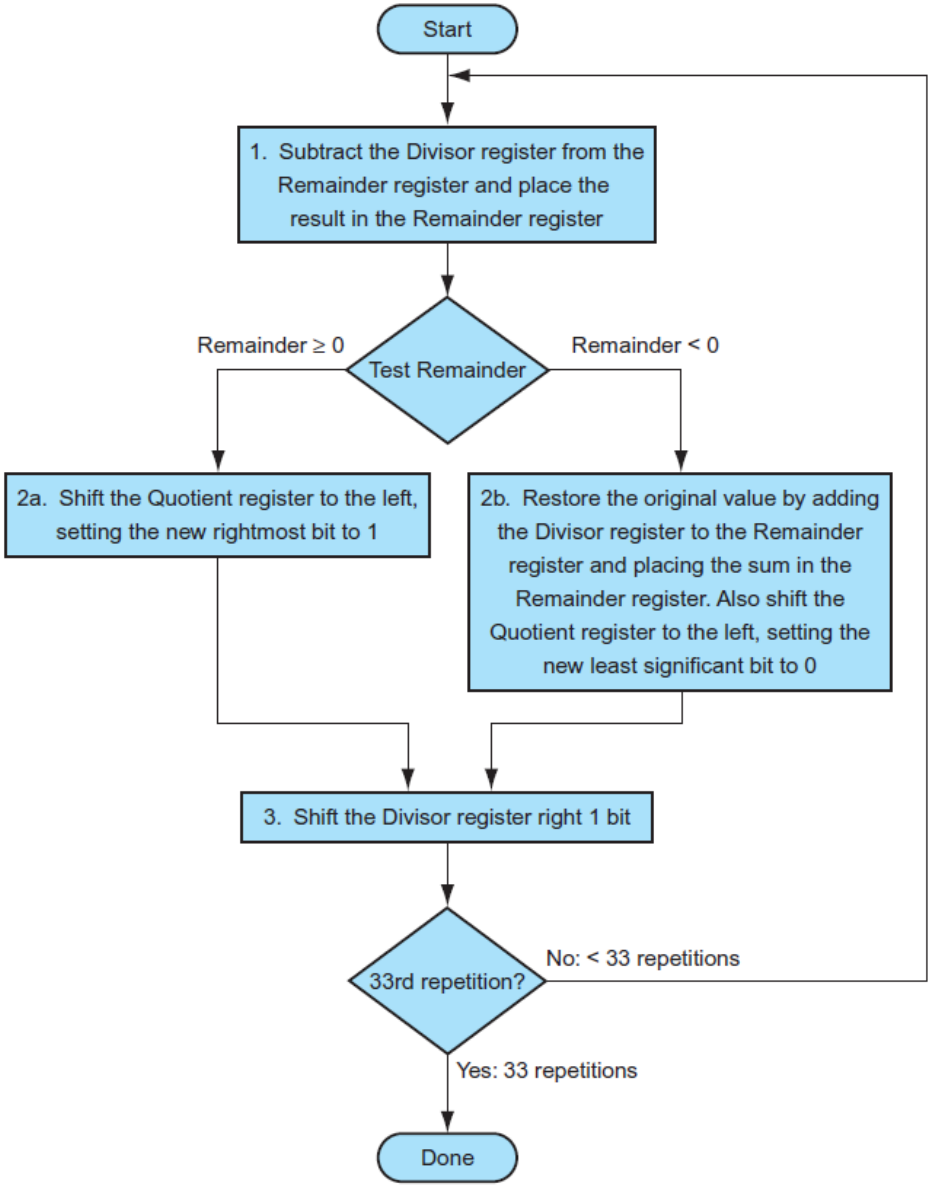


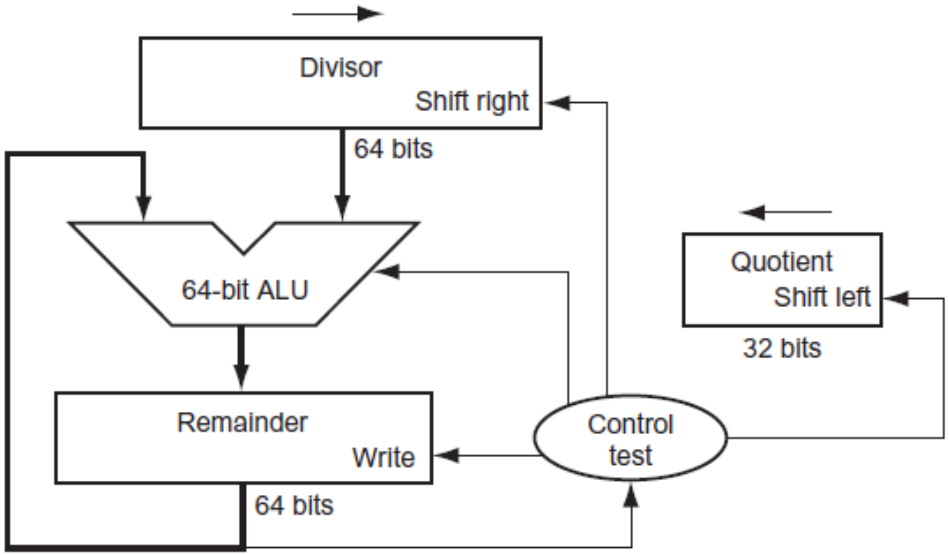
Integer Division: Approach 1



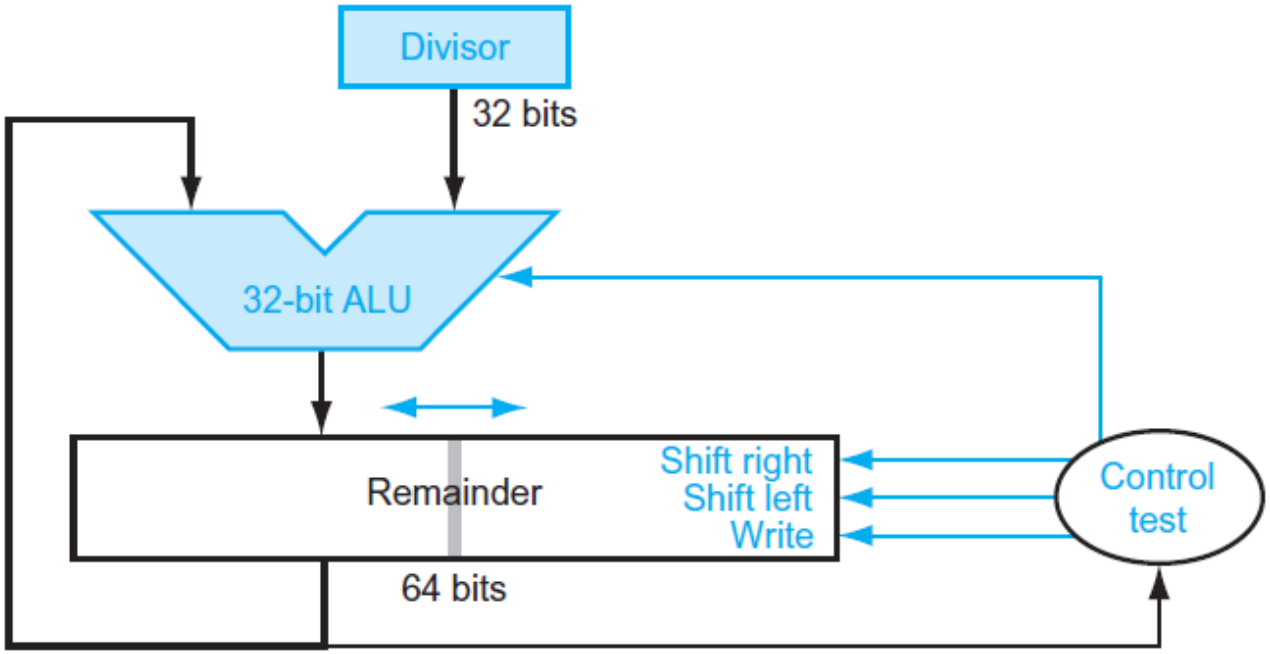
- Check for 0 divisor
- Long division approach
  - If # bits in divisor $\leq$ # in dividend, put 1 in quotient and subtract.
  - Otherwise, put 0 in quotient, bring down next dividend bit
- Restoring division
  - Do the subtract, and if remainder goes < 0, add divisor back
- Signed division
  - Divide using absolute values
  - Adjust sign of quotient and remainder as required

Integer Division: Approach 1





Integer Division: Approach 2



- One cycle per partial-remainder subtraction
- Looks a lot like a multiplier!

  Same hardware can be used for both

MIPS Division

- Use HI/LO registers for result
  - HI: 32-bit remainder
  - LO: 32-bit quotient
- Instructions
  - div rs, rt / divu rs, rt
  - No overflow or divide-by-0 checking

    Software must perform checks if required
  - Use mfhi and mflo to access result

Floating Point

- Representation for non-integer numbers

  Including very small and very large numbers
- Like scientific notation
  - 2.34×10^{56} (normalized)
  - 0.002×10^{4} (not normalized)
  - 987.02×10^{9} (not normalized)
- In binary, $\pm 1.xxxxxxx \times 2^{yyyy}$

  Types float and double in C

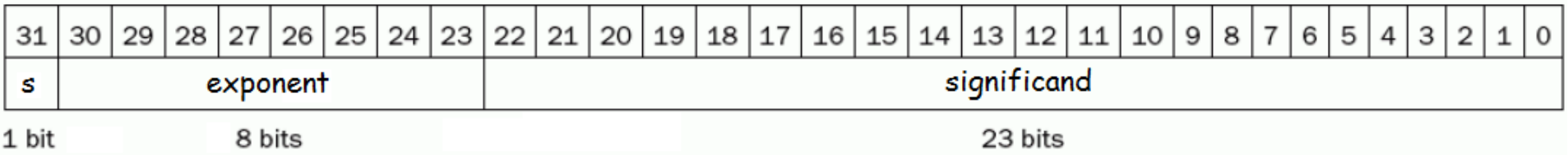
Floating Point Standard

Defined by IEEE Std 754-1985

- Developed in response to divergence of representations
  Portability issues for scientific code
- Now almost universally adopted
- Two representations
  - Single precision (32-bit)
  - Double precision (64-bit)

IEEE Floating-Point Format

Single precision



Exponent (8 bits)	Significand (23 bits)		Equation
	zero	non-zero	
0x00	0, -0	subnormal value	$(-1)^s \times (0.\text{significand}) \times 2^{-126}$
0x01,...,0xFE	normalized value		$(-1)^s \times (1.\text{significand}) \times 2^{\text{Exponent}-127}$
0xFF	∞, $-\infty$	NaN	

IEEE Floating-Point Single Precision Example (1)

s	exponent	significand	hex	representing
1	00000000	00000000000000000000000	0x80000000	-0
0	00000000	00000000000000000000000	0x00000000	0
0	00000000	00000000000000000000001	0x00000001	$0.00\cdots01_{two} \times 2^{-126}$ $= 1.40129846 \times 10^{-45}$
0	00000000	00000000000000000000010	0x00000002	$0.00\cdots10_{two} \times 2^{-126}$ $= 2.80259692 \times 10^{-45}$
0	00000000	11111111111111111111111	0x007FFFFF	$0.11\cdots11_{two} \times 2^{-126}$ $= 1.17549421 \times 10^{-38}$
0	00000001	00000000000000000000000	0x00800000	$1.00\cdots00_{two} \times 2^{(1-127)}$ $= 1.17549435 \times 10^{-38}$
0	11111110	11111111111111111111111	0x7F7FFFFF	$1.11\cdots11_{two} \times 2^{(254-127)}$ $= 3.40282347 \times 10^{38}$
0	11111111	00000000000000000000000	0x7F800000	∞
1	11111111	00000000000000000000000	0xFF800000	$-\infty$

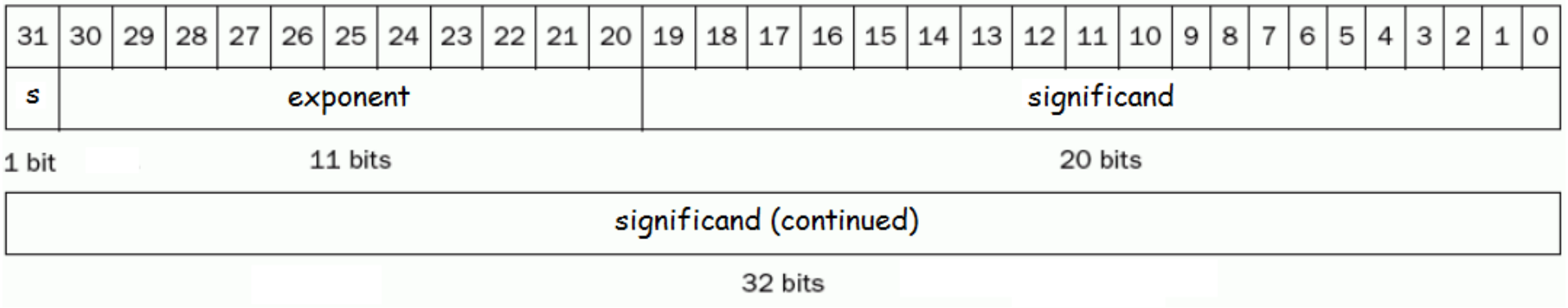
IEEE Floating-Point Single Precision Example (2)

s	exponent	significand	hex	representing
0	11111111	00000000000000000000000	0x7F800000	∞
1	11111111	00000000000000000000000	0xFF800000	$-\infty$
0	11111111	00000000000000000000001	0x7F800001	NaN
1	11111111	00000000000000000000001	0xFF800001	NaN
0	11111111	11111111111111111111111	0x7FFFFFFF	NaN

Floating Point Single Precision

- Exponent [00000000]: subnormal value.
- Exponent [11111111]: infinite value/not a number.
- Smallest value: $\pm 2^{-149} \approx \pm 1.4 \times 10^{-45}$
- (non-infinite) Largest value: $= \pm 1.11\cdots 11_{two} \times 2^{127} \approx \pm 3.4 \times 10^{38}$

Floating Point Double Precision



Exponent (11 bits)	Significand (52 bits)	
	zero	non-zero
0x000	$(-1)^s \times (0.\text{significand}) \times 2^{-1022}$	
0x001,...,0x7FE	$(-1)^s \times (1.\text{significand}) \times 2^{\text{Exponent}-1023}$	
0x7FF	∞, $-\infty$	NaN

- Smallest value: $\pm 2^{-52} \times 2^{-1022} = \pm 2^{-1074} \approx \pm 4.9 \times 10^{-324}$
- (non-infinite) Largest value:
  $= \pm 1.11\cdots 11_{two} \times 2^{1023} \approx \pm 1.79 \times 10^{308}$

Double Precision Range

C Example

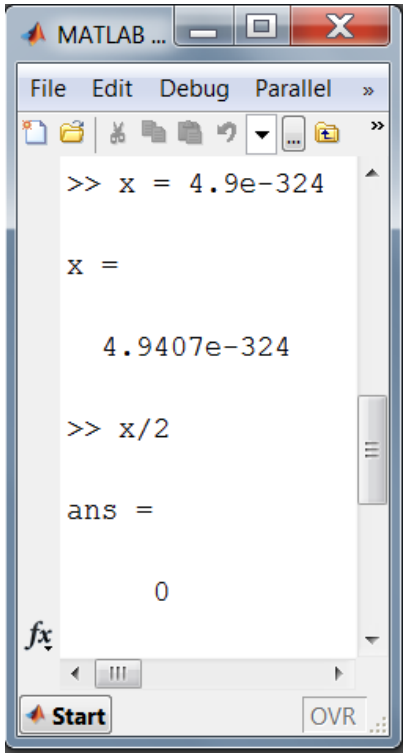
```
main(){
   double x;

   x = 17.9e307;
   printf("x = %e\n",x);

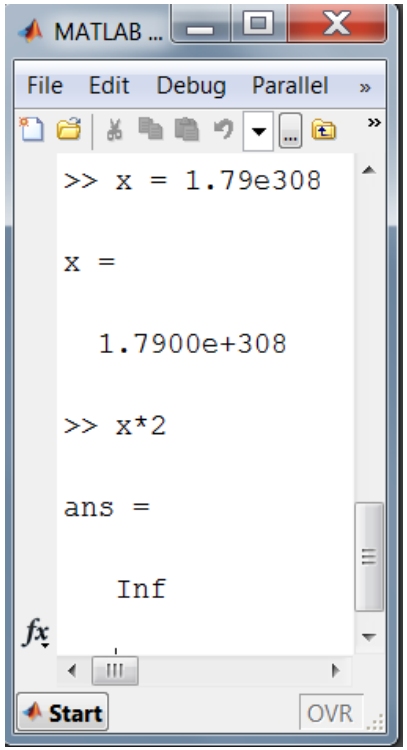
   x = 18e307;
   printf("x = %e\n",x);

}
```

Matlab Lower Limit



Matlab Upper Limit



Fun Fact: Significance of the Significand

Given 52-bit significand, we can keep precision down to 2^{-52}.

- Any significance less than $2^{-52} \approx 2 \times 10^{-16}$ cannot be represented accurately.

Matlab code

```
>> x = 1e17;

>> y = 1e17 + 20;

>> y - x
```

Guess what you'll get.

Fun Fact: Significance of the Significand

Given 52-bit significand, we can keep precision down to 2^{-52}.

- Any significance less than $2^{-52} \approx 2 \times 10^{-16}$ cannot be represented accurately.

Matlab code

```
>> x = 1e17;

>> y = 1e17 + 20;

>> y - x
```

Guess what you'll get.

- Variable y should have 20 more than x, but 52-bit significand would allow upto $x \cdot 2^{-52}$ round off error to happen.
  E.g., in this case $10^{17} \cdot 2^{-52} \approx 22$.
  And, any error upto 22 should be expected.
- Try -x + x + 20 c.f. -x + (x + 20).

Closing note

- อ่าน P&H5e, 3.3-3.5.
- อ่าน บทที่ 3 เอกสารประกอบรายวิชา 188 322 สถาปัตยกรรมคอมพิวเตอร์, COA notes (April 25, 2014).

